

AI Assignment - Report

Sujal Jayant Kumar
DA24B026

April 26, 2026

Overview

The code is implemented in the `src/` folder, with the following Python files:

File	Purpose
<code>feature_extraction.py</code>	Loads the audio files and extract audio features and saves them in test and train <code>pkl</code> files.
<code>utils.py</code>	A utils file just for storing functions that might be used across different files.
<code>GMM.py</code>	GMM class with K-Means initialisation and EM training..
<code>HMM.py</code>	HMM class trained on Baum-Welch Algorithm
<code>train_gmm.py</code>	Loads training features, trains one GMM per digit, saves <code>gmm_models.pkl</code> .
<code>train_hmm.py</code>	Trains one HMM per digit, saves <code>hmm_models.pkl</code> .
<code>evaluate_gmm.py</code>	Runs GMM inference on the test set, prints accuracy and per-digit breakdown.
<code>evaluate_hmm.py</code>	Runs HMM inference on the test set, prints accuracy and per-digit breakdown.
<code>inference.py</code>	Reads a WAV path from python input from terminal and prints GMM and HMM predictions.
<code>compare.py</code>	Evaluates both models on train and test sets, produces all comparison plots.

Feature Extraction

I used a python package called `librosa` to do feature engineering of the audio files. There are a bunch of parameters I used to extract the features properly from the audio files.

- **TARGET_SR** = 16,000: Every audio file is converted so that the sound is measured 16,000 times per second (16kHz), no matter how it was originally recorded. This leaves out the possibility of different sampling of audio files and this might make feature making inconsistent.

- **N_MFCC** = 13: I used Mel-Frequency Cepstral Coefficients (MFCCs) to compress the spectrum of each recording in a small number of coefficients. This ranges from low coefficient (slow, coarse) to high coefficient (fast, finer).
- **N_MELS** = 40: Before computing MFCC, we will divide our spectrum of audio in 40 Frequency Bands based on the Mel Scale. After this is done, we can compute each band's MFCC and then select first 13 of them (which is N_MFCC). The choice of keeping 13 MFCCs is empirical, in my experiments, using the first 13 coefficients gave noticeably better performance.
- **WIN_MS** = 25: Each audio file is cut into parts of 25 milliseconds, and then the audio is analyzed; 25 milliseconds is short enough to capture 1 phoneme across the audio file.
- **HOP_MS** = 10: Each of the 25 ms windows is separated by 10 ms. Because of the smaller hop, we can capture more frames. Now, I can approximately capture 47 frames for a 0.5 sec recording, thus adding more detail to the pronunciation.
- **PRE_EMPH** = 0.97: Before any analysis, a high-pass filter is applied: $y[t] \leftarrow y[t] - 0.97 \cdot y[t - 1]$. Pre-emphasis boosts the high-frequency part before the FFT, making all parts of the spectrum contribute roughly equally to the MFCCs.

Next we do the feature extraction and also split the data in training and test data.

- The WAV file is loaded and sampled according to TARGET_SR (16000) and then high-pass filter is ran (I described it above).
- `librosa.feature.mfcc()` is called with the given parameters and divides each of the audio files into multiple overlapping frames. For each of the bands after doing audio analysis, (using MFCC), first 13 (N_MFCC) are selected. The output is a matrix of size (13, T) where T is the number of frames from our recording.
- Two more features are added to this matrix, Δ and $\Delta\Delta$ which means how fast is each MFCC changing (Δ) while $\Delta\Delta$ captures the rate of change of the Δ .
- This gives us another 2 features each frame which is (39, T) sized matrix features. Transposed to (T, 39)
- Each of the 39 feature dimensions is centred and scaled:

$$\mathbf{x}_t \leftarrow \frac{\mathbf{x}_t - \boldsymbol{\mu}}{\boldsymbol{\sigma} + \epsilon}$$

where $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ are the mean and standard deviation computed over all frames in the file.

Finally after this we end up with a matrix of size (T, 39) for each WAV file passed (T is number of frames). These matrices are stored as dictionaries in the pkl files. For example:

`train_data['k']`: a Python list of $(T_i, 39)$ arrays, one per training file for digit k .

We just split the train and test data based on the numbering on the file name where 1, 2, 3, 4 and 5 go to `train_data.pkl` while 6 goes to `test_data.pkl` where these numbers mean the index of speaker.

utils.py

I just made this file to encapsulate the functions I will use in multiple files.

log_of_sum_exp()

: Computes $\log(\sum_i e^{a_i})$ without arithmetic overflow or underflow. In both GMM and HMM probabilities are very tiny numbers. Multiplying this leads it to very close to 0, working in

log-space converts multiplications to additions, but adding logs requires $\log(\sum e^{a_i})$. If all a_i are very negative, e^{a_i} underflows to zero before the sum. Specifically for HMMs, I faced this issue a lot, this also handles another case where every input is $-\infty$ leading to issues and numpy returning NaN, again faced in HMM.

$$\log \sum_i e^{a_i} = \max_i(a_i) + \log \sum_i e^{a_i - \max_i(a_i)}$$

VectorQuantizer

This class is only used by HMM, It converts the continuous 39-dimensional frames into integer symbol indices, converting a sequence to into a sequence of tokens that the HMM's emission matrix can work with. It has two functions, **fit** and **quantize**.

- **fit**: Take all training frames and assign it to a centroid, basically we run K Means Clustering, first by selecting random centroids and then assigning it to clusters and keep updating centroids. This repeats for up to `max_iter = 20`. iterations or until the centroids stops changing.
- Given a $(T, 39)$ feature matrix, replaces each frame with the index of the nearest centroid. The output is a 1-D integer array of length T , where each element is a number from 0 to 63. These indices/labels can be used by HMM to model.

plot_confusion_matrix()

This function just that takes a 10×10 integer confusion matrix and saves as PNG file.

GMM.py

In this python file, GMM is a class implemented. The GMM assumes data generation by K different Gaussians, each handling a different type of frame. GMM ignores the time order; all frames from the training data are pooled together and given to GMM. This means "ze-ro" and "ro-ze" have same probabilities.

- **initial parameters**: (all of these hyperparameters, these seem to run in small duration with good results)
 - `n_gaussian = 8`: number of mixture components.
 - `max_iter = 100`: maximum EM iterations per restart.
 - `tol = 1e-4`: convergence tolerance on $|\Delta \log L|$.
 - `n_restarts = 3`: number of independent random restarts.
 - `reg_covar = 1e-3`: minimum variance floor.
- **_kmeans_centre and _kmeans_run**: Initially, rather than starting randomly, we pick centres using K-Means clustering. Runs 20 iterations of (1) assign every point to its nearest centroid, (2) move each centroid to the mean of its assigned points.
- **_em_start()** Initialises the three GMM parameter sets from the K-means solution:
 1. **Means** μ_k : set to the K-means centroid of cluster k .
 2. **Weights** π_k : set proportional to the cluster size.
 3. **Covariances** σ_k^2 : set to the per-feature variance within cluster k , with a floor of `reg_covar` to prevent zero-variance collapse.

- **_log_gauss** Computes the log-probability density of every data point under every Gaussian component:

$$\log \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \text{diag}(\boldsymbol{\sigma}_k^2)) = -\frac{1}{2} \left[D \log(2\pi) + \sum_{d=1}^D \log \sigma_{kd}^2 + \sum_{d=1}^D \frac{(x_{nd} - \mu_{kd})^2}{\sigma_{kd}^2} \right]$$

- **_e_step()** Computes the *responsibility* r_{nk} = the probability that data point n was generated by component k :

$$r_{nk} = \frac{\pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

- **_m_step** Given the responsibilities, updates all parameters to maximise the expected complete-data log-likelihood:

$$\begin{aligned} N_k &= \sum_{n=1}^N r_{nk} && \text{(effective count for component } k) \\ \pi_k &\leftarrow \frac{N_k}{N} \\ \boldsymbol{\mu}_k &\leftarrow \frac{\sum_n r_{nk} \mathbf{x}_n}{N_k} \\ \sigma_{kd}^2 &\leftarrow \frac{\sum_n r_{nk} (x_{nd} - \mu_{kd})^2}{N_k} \end{aligned}$$

- **_fit_iter()** Calls **_em_start**, then alternates E-step and M-step until $|\log L_t - \log L_{t-1}| < \text{tol}$ or **max_iter** is reached. Returns the final log-likelihood.
- **fit()**: Runs **n_restarts** = 3 independent calls to **_fit_iter()**.
- **score()** The model calculates the probability of every individual audio frame and adds them all together. We then divide that total by the number of frames (T) to get an average per-frame score. This step is crucial because it ensures that long recordings (slow speakers) and short recordings (fast speakers) are judged fairly on the exact same scale.

$$\text{score}(X) = \frac{1}{T} \sum_{t=1}^T \log \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_t | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

1 HMM.py

The HMM solves this by first compressing each frame to one of 64 integer symbols (via the Vector Quantizer) and then modelling the resulting symbol sequence. The emission distribution becomes a simple 5×64 table of categorical probabilities

- **initial parameters:**
 - $S = 5$ hidden states (hyper parameter, 5 seemed to give good accuracy)
 - $M = 64$ possible observation symbols (the VQ centroid size).
 - Initial state distribution $\boldsymbol{\pi}$ always starts in state 0.
- **_bm_start()**
 1. **Initial state $\boldsymbol{\pi}$:** $\pi_0 = 1$, all others 0.
 2. **Transition matrix $\mathbf{A}$:** $A_{i,i} = 0.5$, $A_{i,i+1} = 0.5$ for $i < S - 1$; $A_{S-1,S-1} = 1.0$.
 3. **Emission matrix $\mathbf{B}$:** initialised to a near-uniform distribution with small random perturbations (so symmetry is broken and different states learn different patterns).
- **_log_emission** Evaluates the log-probability of observing symbol X_t in state j at time t .

- **forward** Computes $\log \alpha_t(j)$ where $\alpha_t(j) = P(x_1, x_2, \dots, x_t, q_t = j | \lambda)$ – the probability of having seen the first t observations *and* being in state j at time t , where

$$\lambda = (A, B, \pi)$$

, the current state of the model.

- **backward** Computes $\log \beta_t(i)$ where $\beta_t(i) = P(x_{t+1}, \dots, x_{T-1} | q_t = i, \lambda)$ – the probability of the remaining observations given that we are in state i at time t .

Recursion (run backwards from $t = T - 2$ to $t = 0$):

$$\log \beta_t(i) = \log \sum_{j=0}^{S-1} e^{\log A_{ij} + \log B_{j, x_{t+1}} + \log \beta_{t+1}(j)}$$

Initialisation: $\log \beta_{T-1}(i) = 0$ for all i .

- **fit() - BW Algo (EM) E-step:**

1. Run forward to get $\log \alpha$.
2. Run backward to get $\log \beta$.
3. Compute state-occupation probability (gamma): $\log \gamma_t(i) = \log \alpha_t(i) + \log \beta_t(i) - \log P(X|\lambda)$
4. Calculates the exact probability of the model transitioning from State i at time t , to State j at time $t + 1$.S

$$\log \xi_t(i, j) = \log \alpha_t(i) + \log A_{ij} + \log B_{j, x_{t+1}} + \log \beta_{t+1}(j) - \log P(X|\lambda)$$

5. Accumulate counts for the emission matrix **B**: for each time step t , add $\gamma_t(j)$ to the count for symbol x_t in state j .

M-step:

$$A_{ij} \leftarrow \frac{\sum_s \sum_t \xi_t^{(s)}(i, j)}{\sum_s \sum_t \gamma_t^{(s)}(i)}$$

$$B_{j,m} \leftarrow \frac{\sum_s \sum_{t: x_t^{(s)}=m} \gamma_t^{(s)}(j)}{\sum_s \sum_t \gamma_t^{(s)}(j)}$$

For calculating the score, it runs the forward algorithm on the test sequence and returns the log-likelihood divided by the sequence length.

Other python files

- **inference.py** Reads a WAV file path from standard input, extracts features, and runs both GMM and HMM inference in sequence, printing both predictions.
- **train_gmm.py** and **train_hmm.py**: Both of them load the **train_data.pkl** and, for each digit, pool all training feature matrices in a matrix and pass it to **GMM.fit()**, storing the models in their respective model files. For HMM, though the first process is to convert all 2500 training sequences into one big matrix and training the **VectorQuantizer** to 64 centroids.
- **evaluate_gmm.py** and **evaluate_hmm.py** Loads **test_data.pkl** and **gmm.models.pkl**. For each of the 500 test files, scores it against all 10 digit models and predicts the digit with the highest score. Reports overall accuracy and per-digit accuracy.
- **compare.py** Evaluates both models on both train and test splits to compute validation and test accuracies. Generates:
 1. The GMM confusion matrix (saved as **gmm_cm.png**).
 2. The HMM confusion matrix (saved as **hmm_cm.png**).
 3. A per-digit accuracy line chart (saved as **per_digit_comparison.png**).

Results

Confusion Matrices

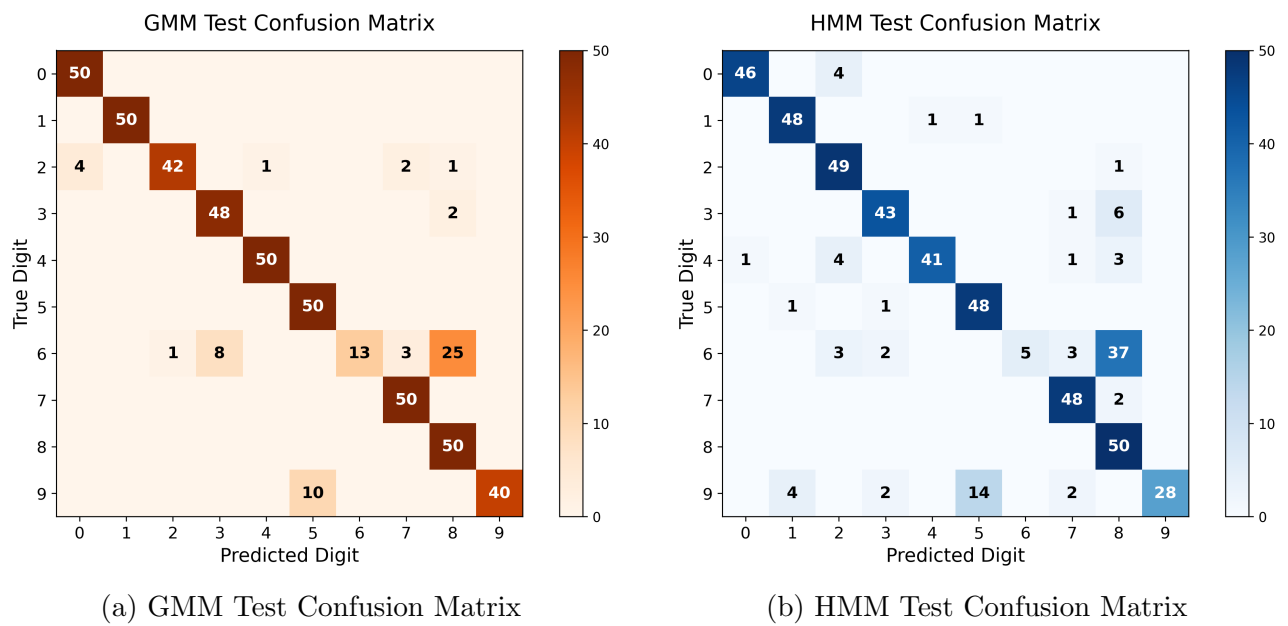


Figure 1: Confusion matrices for GMM (left) and HMM (right) on the test split (speaker 6).

Per-Digit Accuracy

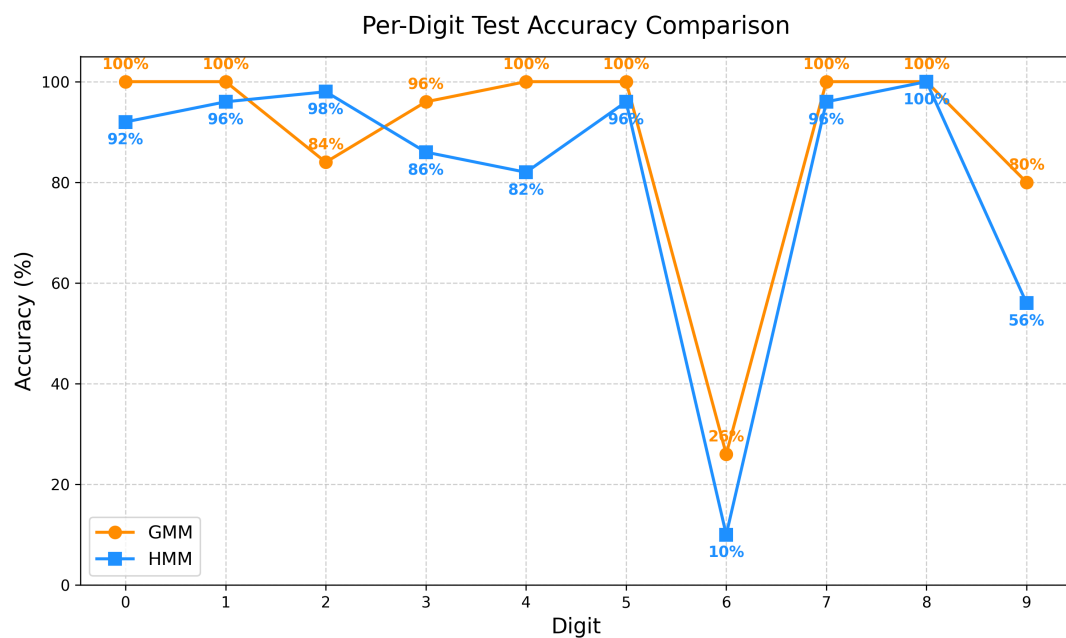


Figure 2: Per-digit test accuracy for GMM (orange) and HMM (blue).

Overall Accuracy

Model	Validation Accuracy	Test Accuracy
GMM	96.20	88.60
HMM	92.64	82.60

Observations

- The digit 6 is misclassified the most in test data, achieving only 26% for GMM and 10% for HMM. Most “six” samples are misclassified as “eight” or “five”.
- The digit 9 reaches 80% with GMM but only 56% with the HMM. The HMM confusion matrix shows many “nine” samples being labelled as “five”.
- The digit 2 is the only digit where HMM performs better than GMM. GMM misclassified 2 as “one” and “seven”.
- At the digits 0, 1, 4, 5, 7 and 8, both models perform really good having a good range of 80-100 in accuracy for the test data.
- GMM beats HMM in both Validation and Test Accuracy.

Inferences

- GMM works directly with the 39 Parameters we extracted from the audio that has the most information, while HMM discards a lot of data during centroid calculation initially for vector quantization. With only 64 centroids, two slightly different things would start sounding similar maybe like “nine” and “five”. (One observation above mentioned about low accuracy of “nine” in HMM.). Accuracy can be increased by number of centroids like 128, or more, but again it will take a lot of time.
- HMM can work better than GMM given it tracks which phoneme comes in which order. Maybe with longer recordings, like sentences, HMM would end up performing better than GMM.
- But for the current dataset with small recordings, GMM performs better than HMM, though both perform pretty well having a reasonable score.
- HMM might take a little more time also given it has to first extract the centroids first during the training period which adds another overhead in performance.

Run the Code

1. `feature_extraction.py` - Will create `test_data.pkl` and `train_data.pkl`
2. `train_gmm.py` - Fits GMM on data, creates `gmm_models.pkl`
3. `train_hmm.py` - Fits HMM on data, creates `hmm_models.pkl`
4. `evaluate_gmm.py` - Evaluates the GMM models and gives the results
5. `evaluate_hmm.py` - Evaluates the HMM models and gives the results.
6. `compare.py` - Compares the two models and output results and saves 3 plots, `gmm_cm.py`, `hmm_cm.py`, `per_digit_comparision.py`
7. `inference.py` - It will ask you the path of the audio file, processing it, it will give results.